

Hashing is a fundamental technique in cryptography that transforms an input message of any length into a fixed-size string of characters, called a hash value (informally called hash) or message digest (informally called digest). A good hash function is designed to be fast to compute while being practically impossible to reverse, meaning the original message cannot be reconstructed from its hash value. It also ensures that even a small change in the input—such as altering a single character—produces a completely different hash, a property known as the avalanche effect. Examples for hash functions include SHA-256, SHA-384, SHA-512, BLAKE2, BLAKE3, MD5, etc. Hash functions are widely used to ensure data integrity, verify passwords, generate digital signatures, and support many other security protocols.

Let us look at a simple example of a hash function. Suppose we want to hash numbers between 1 and 1000 using the following toy hashing algorithm. First, we compute the sum of the digits of the number. If the result has more than one digit, we repeat the process—adding the digits again—until we obtain a single-digit number. This final single-digit value will be the hash value. Consider the number 123. The sum of its digits is: $1 + 2 + 3 = 6$. Since 6 is already a single-digit number, the hash value of 123 is 6. Now take the number 987. The first step gives: $9 + 8 + 7 = 24$. Because 24 has more than one digit, we apply the step again: $2 + 4 = 6$. So the hash value of 987 is 6.

Although this toy hash algorithm is very simple, it highlights three important characteristics of a good hash function. First, the hash value (also called a message digest or digest) has a fixed size, which in this example is a single digit. Second, it is not possible to recover the original message from the hash value. For instance, a hash value of 6 does not reveal whether the input was 123 or 987. Third, different inputs can produce the same hash value. Our earlier examples demonstrate this: both 123 and 987 yield a hash value of 6. Such a situation is known as a hash collision.

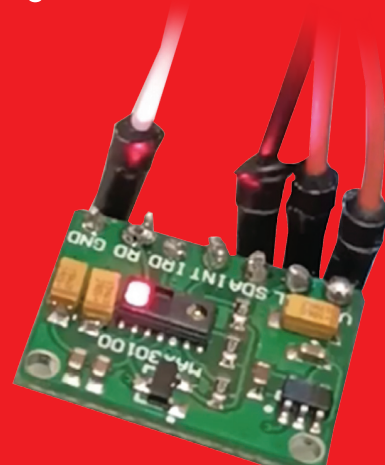
A hash collision occurs when two different inputs produce the same hash value under a given hash function. Because hash functions map data of arbitrary size to a fixed-size output, collisions are theoretically unavoidable. However, good cryptographic hash functions are designed so that collisions are extremely rare and computationally infeasible to find. It is also evident that increasing the size of the hash value reduces the likelihood of collisions. For instance, in our toy example, if we allowed the hash value to be two digits instead of one, the results would differ. The hash of 123 would be 06, while the hash of 987—after a single round of digit-sum computation—would be 24. With more possible output values available, the chance of two inputs landing on the same hash value becomes smaller.

Now, let us try to understand how hashing can be used for message authentication. Message authentication is primarily used to prove the integrity of a message,

IOT DIY KIT: Health Tech & Medical Devices

Want to create exciting IoT projects related to medical electronics, medical devices, health or wellness tech?

This DIY kit lets you build smart IoT projects with WiFi connectivity and built-in sensors for monitoring, alerts, and tracking.



POWERED BY



**IndusBoard
COIN**

Super small sized development board that enables you to create solutions for many use-cases where small size is super important.



SPECIAL INTRO PRICE

₹2,049 (Inclusive of GST)

KNOW MORE / BUY NOW: efy.co.in/healthiotkit

Developed by the Electronics For You team, for you.

DEALERS: • Om Technology Centre, Bengaluru (Mob: +919845400864)
• Bharathi Electronics: Coimbatore (Mob: +919952228618)